CP317 - Software Engineering

Project Title: Car Rental System

Deliverable: Sprint 1

Group ID: Team # 28

Team Members and Roles:

- SabilUllah Hussaini (Product Owner)
- Sahil Sandhu (Product Owner & Developer)
- Moosa Ahmed (Scrum Master)
- Mahad Farrukh (Developer)
- Ayesha Raheel (Developer)
- Kritika Kamath (Developer)
- Shalin Panjwani (Developer)

Sprint Goal (½ Page)

Sprint Objective

The goal of Sprint 01 is to deliver a functional authentication system that includes both login and registration features connected to a working database. This core functionality ensures that the project's foundation is stable and secure, allowing for user-specific access and interaction with the system.

Prioritization Rationale

The selected backlog items (AUTH-1, AUTH-2, UI-1, UI-2) were prioritized for this sprint because they represent the **absolute foundation** of the entire car rental system:

1. **Authentication is Prerequisite**: User accounts must exist before any personalized features (bookings, history, profiles) can be implemented. Without authentication, the system cannot track user-specific data.
2. **Search is Core Value**: The ability to discover available vehicles is the primary reason users will visit the platform. Without search functionality, the system provides no immediate utility to customers.
3. **Sequential Dependency**: These stories build upon each other logically, users must register before logging in, and must be able to search before they can view vehicle details or make reservations.
4. **Technical Foundation**: Successfully implementing these stories validates our entire technology stack (SQLite, , C#) and establishes the architectural patterns for all future development.
5. **Risk Mitigation**: By tackling the most complex integration challenges first (database connections, API routes, frontend-backend communication), we reduce technical risk early in the project lifecycle.

This focused approach ensures we deliver a **working, testable prototype** that demonstrates the core user journey while establishing a robust technical foundation for subsequent sprints.

Sprint Backlog (1–2 Pages + Excel File)

Summary

During Sprint 01, the team successfully implemented the Login and Registration modules, established the database connection, and tested basic user flows between the UI and backend.

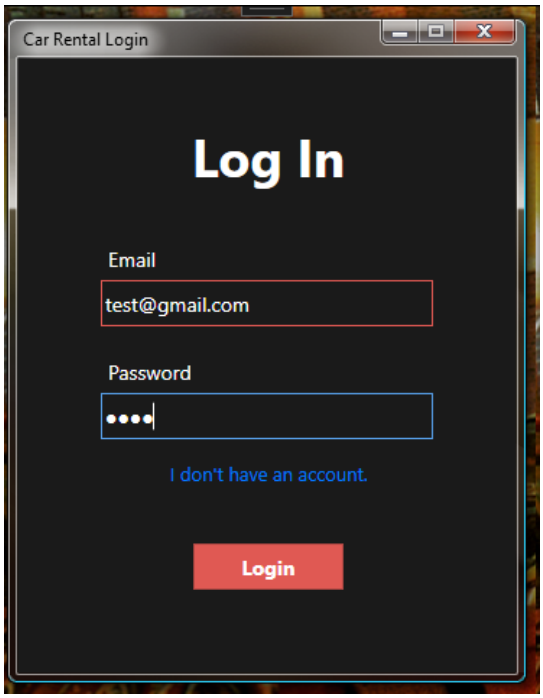
Key Implementations

- Login and Registration pages developed using WPF,C#,etc
- Backend configured with Entity , Framework,etc
- Data securely stored in Sql lite,etc

Story ID	Story Title	Story Points	Priority	Assigned Team Member(s)	Tasks	Status
AUTH-1	Secure Account Creation	3	High	Mahad,Ayesha,Sahil, Sabil	• Database Schema Design (Mahad) • Registration Form UI (Ayesha) • Password Hashing Implementation (**Sahil**) • Security Review (Sabil)	Done
AUTH-2	Secure Login & Logout	2	High	Ayesha,Sahil,Moosa, Kritika	• Login Form UI (Ayesha) • Session Management (**Sahil**) • Logout Functionality (Moosa) • UI Testing (Kritika)	Done
UI-1	Vehicle Search	3	High	Kritika,Shalin,Sahil, Mahad	• Search Form UI (Kritika) • Search Backend Logic (Shalin) • Date/Location Validation (**Sahil**) • Database Integration (Mahad)	Done
UI-2	Vehicle Search Result Basic Information	5	High	Sahil, Shalin, Moosa, Sabil, Ayesha, Kritika	• Results Page UI (Shalin) • Vehicle Data Display (Moosa) • Performance Optimization (**Sahil**) • Image Handling (Sabil) • Responsive Design (Ayesha)	Done

Code Snippet Example & Screenshots

Login Screen:

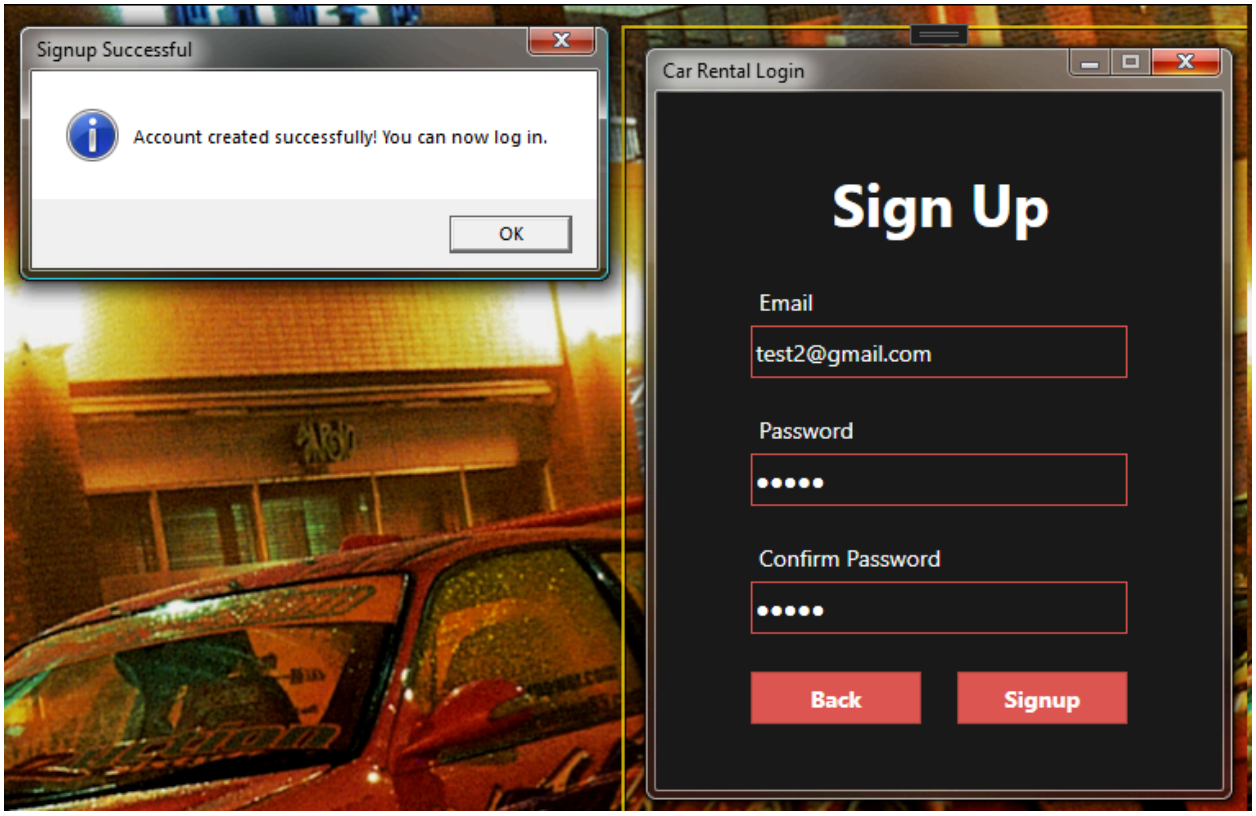



```
1 reference
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
    var email = EmailTextBox.Text;
    var password = PasswordTextBox.Password;

    using (UserDataContext context = new UserDataContext())
    {
        var userFound = context.Users.FirstOrDefault(user => user.Email == email && user.Password == password);
        if (userFound != null)
        {
            GlobalVariables.accountID = userFound.AccountID;
            var userInfo = context.UserInformations.Find(GlobalVariables.accountID);

            if (userInfo == null)
            {
                OpenSetupWindow();
            }
            else
            {
                GlobalVariables.loggedIn = true;
                OpenMainWindow();
            }
        }
        else
        {
            MessageBox.Show("Invalid email or password. Please try again.", "Login Failed", MessageBoxButton.OK, MessageBoxImage.Warning);
        }
    }
}
```

The function is executed when the login button is clicked. The information in the Email and Password TextBoxes are compared to the Email and Password columns of the User table in SQLite. Once the user is found, it checks if another table, UserInformations, is empty. If it is

empty it opens the Profile Setup screen where the user must enter their personal information to continue.

Sign Up Screen:

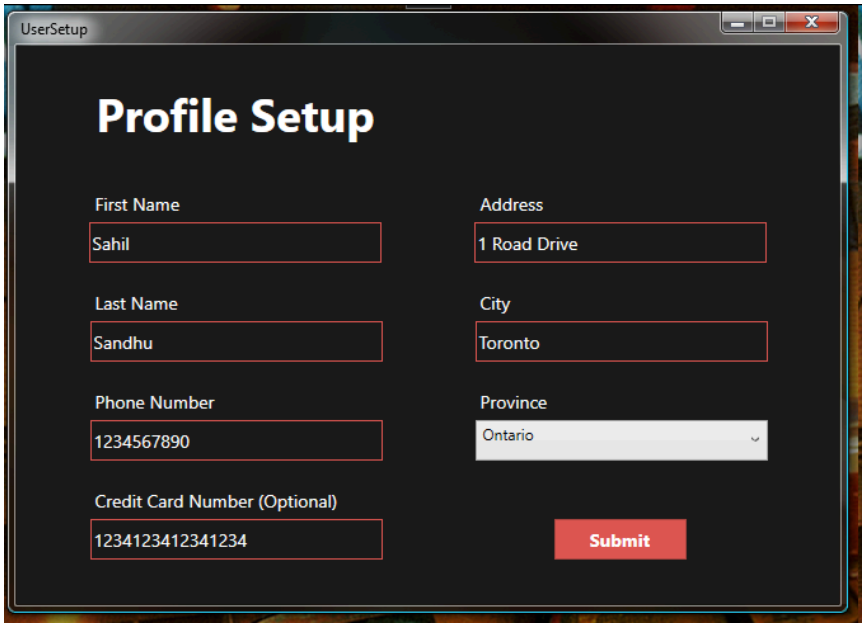



```
1 reference
private void SignupButton_Click(object sender, RoutedEventArgs e)
{
    if(EmailTextBoxSignup.Text.Contains("@") && PasswordTextBoxSignup.Password == PasswordConfirmTextBoxSignup.Password && PasswordTextBoxSignup.Password.Length > 0)
    {
        using (UserDataContext context = new UserDataContext())
        {
            var newUser = new User
            {
                Email = EmailTextBoxSignup.Text,
                Password = PasswordTextBoxSignup.Password
            };
            context.Users.Add(newUser);
            context.SaveChanges();
        }
        MessageBox.Show("Account created successfully! You can now log in.", "Signup Successful", MessageBoxButton.OK, MessageBoxImage.Information);
        LoginCanvas.Visibility = Visibility.Visible;
        SignupCanvas.Visibility = Visibility.Hidden;
    }
    else if (PasswordTextBoxSignup.Password.Length == 0)
    {
        MessageBox.Show("You must enter a password.", "Signup Failed", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    else if(PasswordTextBoxSignup.Password != PasswordConfirmTextBoxSignup.Password)
    {
        MessageBox.Show("The entered passwords do not match.", "Signup Failed", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
    else if (EmailTextBoxSignup.Text.Contains("@") == false)
    {
        MessageBox.Show("The entered email is not valid.", "Signup Failed", MessageBoxButton.OK, MessageBoxImage.Warning);
    }
}
```

When the Signup button is clicked, all of the TextBoxes are checked to ensure their content is valid. Each invalid textbox has its own error message. If everything is valid, the TextBox

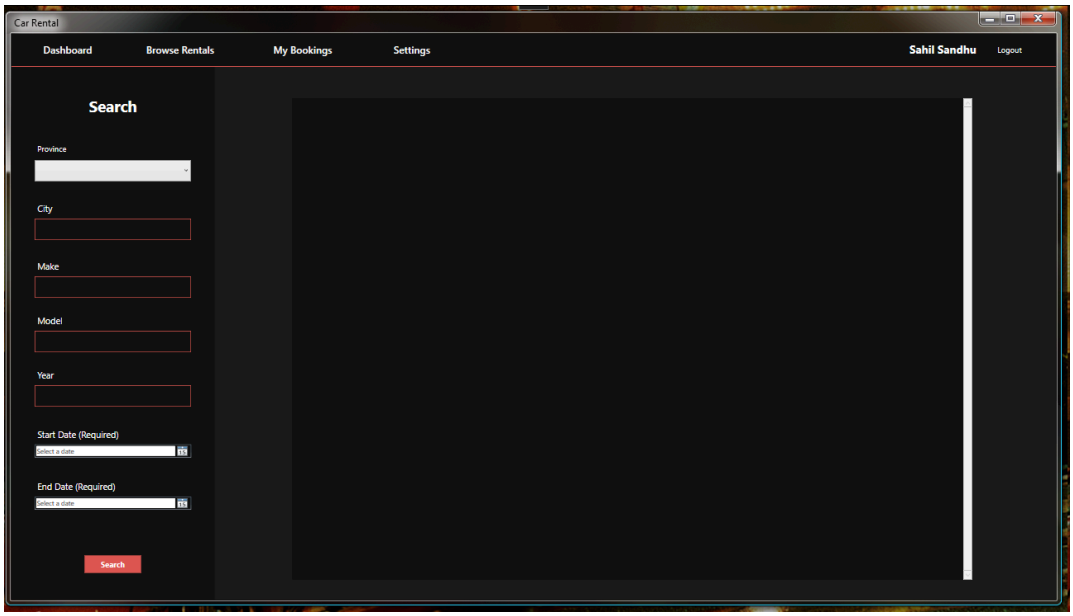
information is stored in the User table in SQLite. Each user has its own individual ID to prevent duplicate accounts.

Profile Setup Screen:

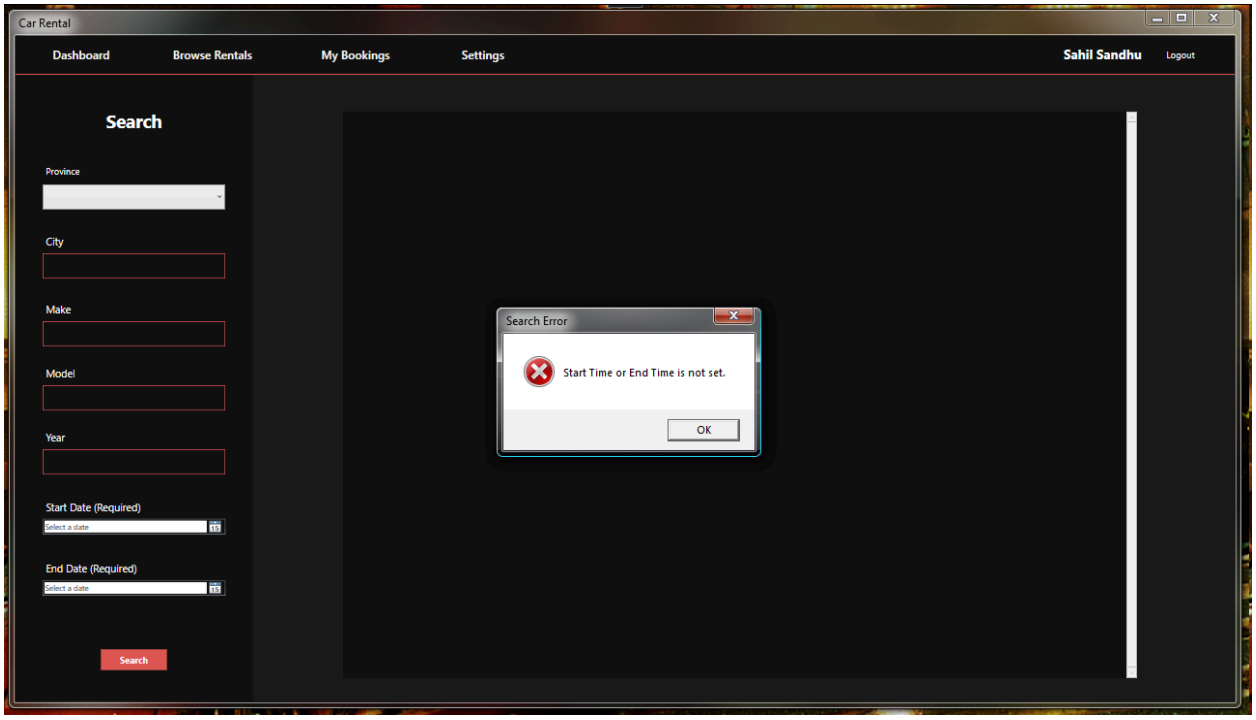


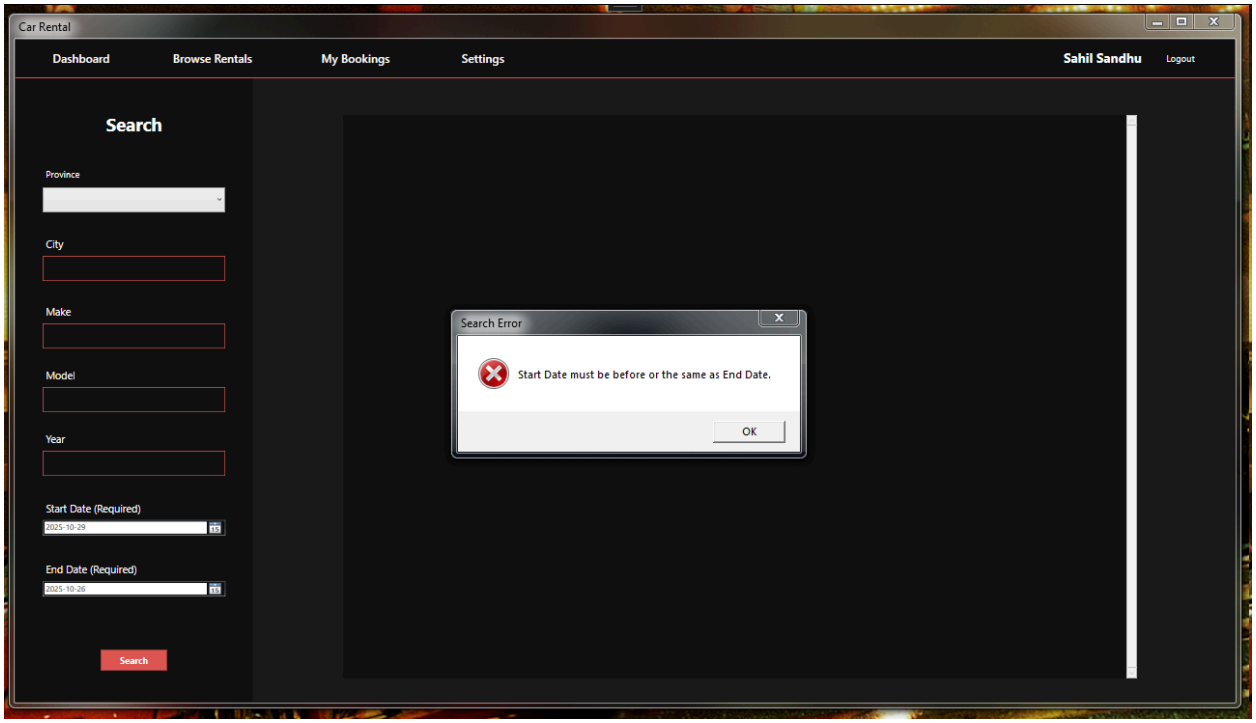
Similar to the Sign Up screen in functionality.

Browse Rentals Screen:



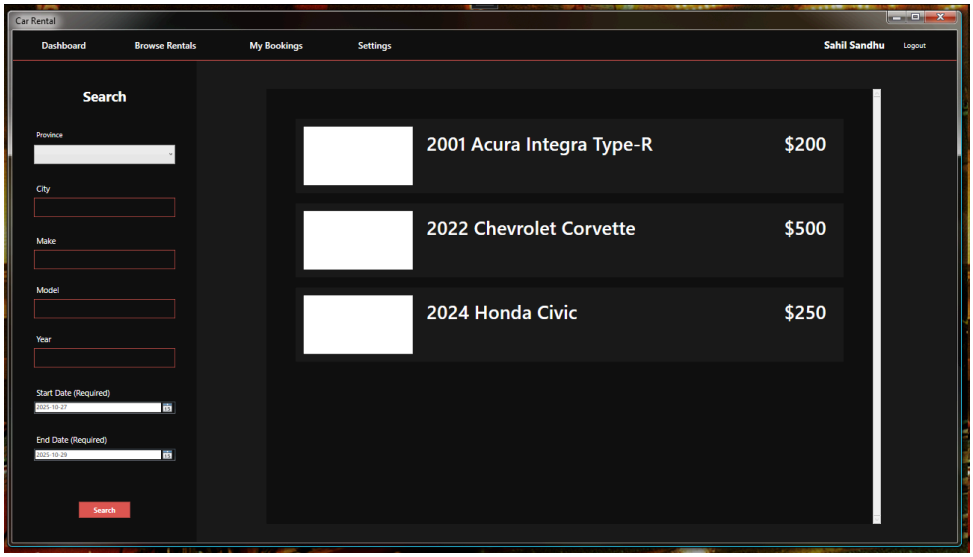
Invalid Search Handling:



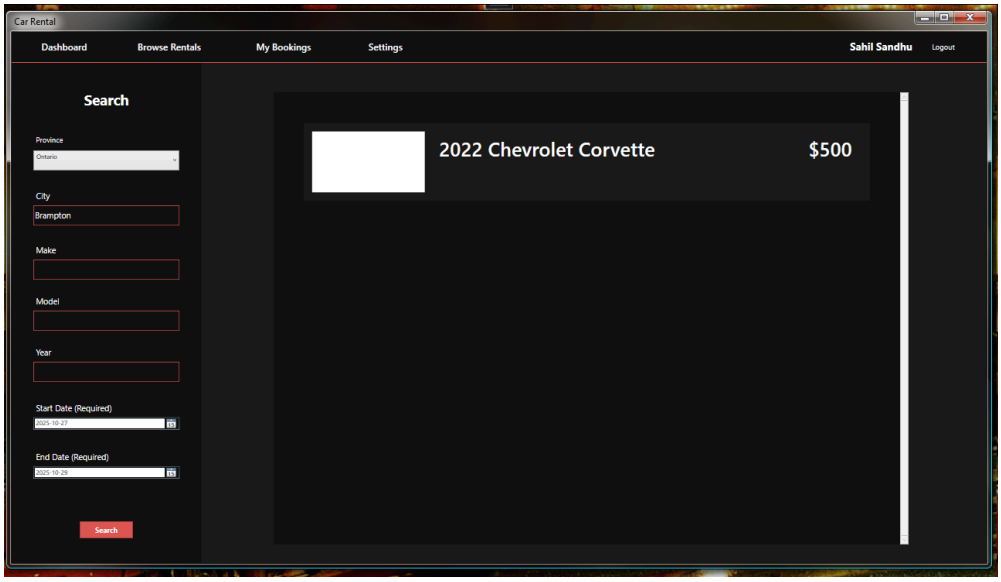


SQL CarEntry Table:

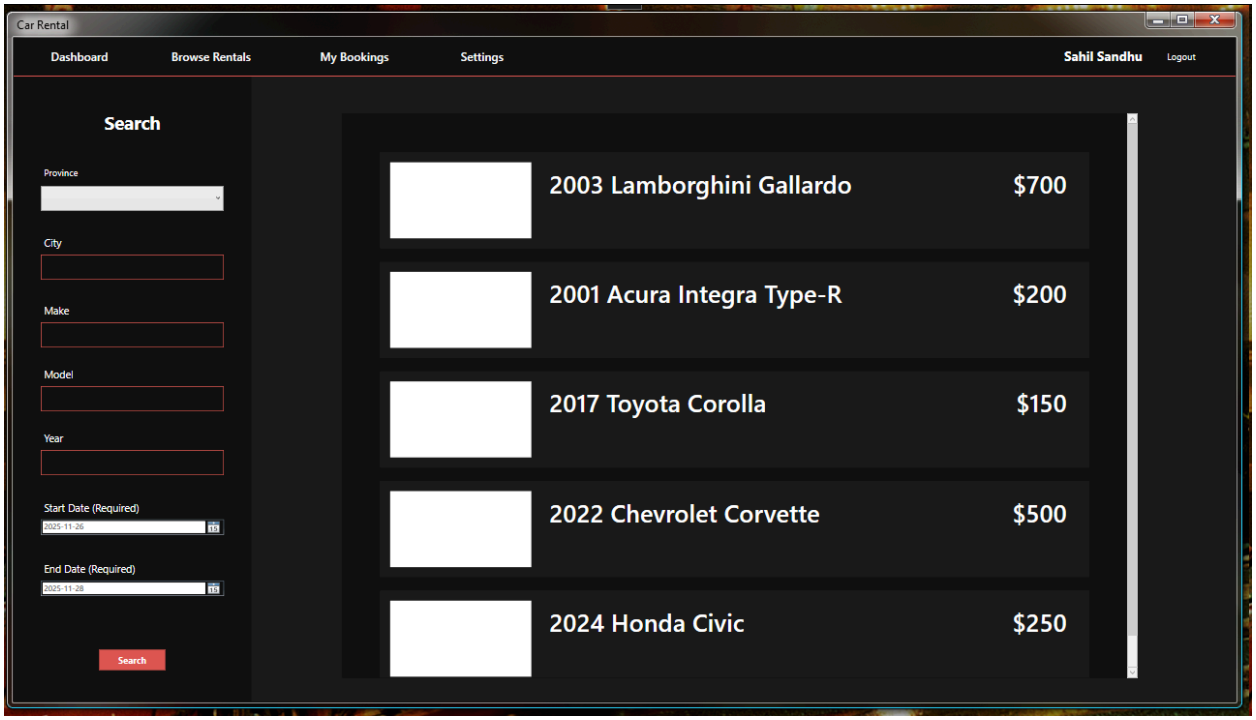
	CarID	CarYear	CarMake	CarModel	City	Province	Price	ImageUrl	IsAvailable	AvailabilityDate
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	2003	Lamborghini	Gallardo	Brampton	Ontar…	700		0	2025-10-30
2	2	2001	Acura	Integra Type-R	Montreal	Quebec	200		1	
3	3	2017	Toyota	Corolla	Toronto	Ontar…	150		0	2025-11-15
4	4	2022	Chevrolet	Corvette	Brampton	Ontar…	500		1	
5	6	2024	Honda	Civic	Vancouver	Briti…	250		1	



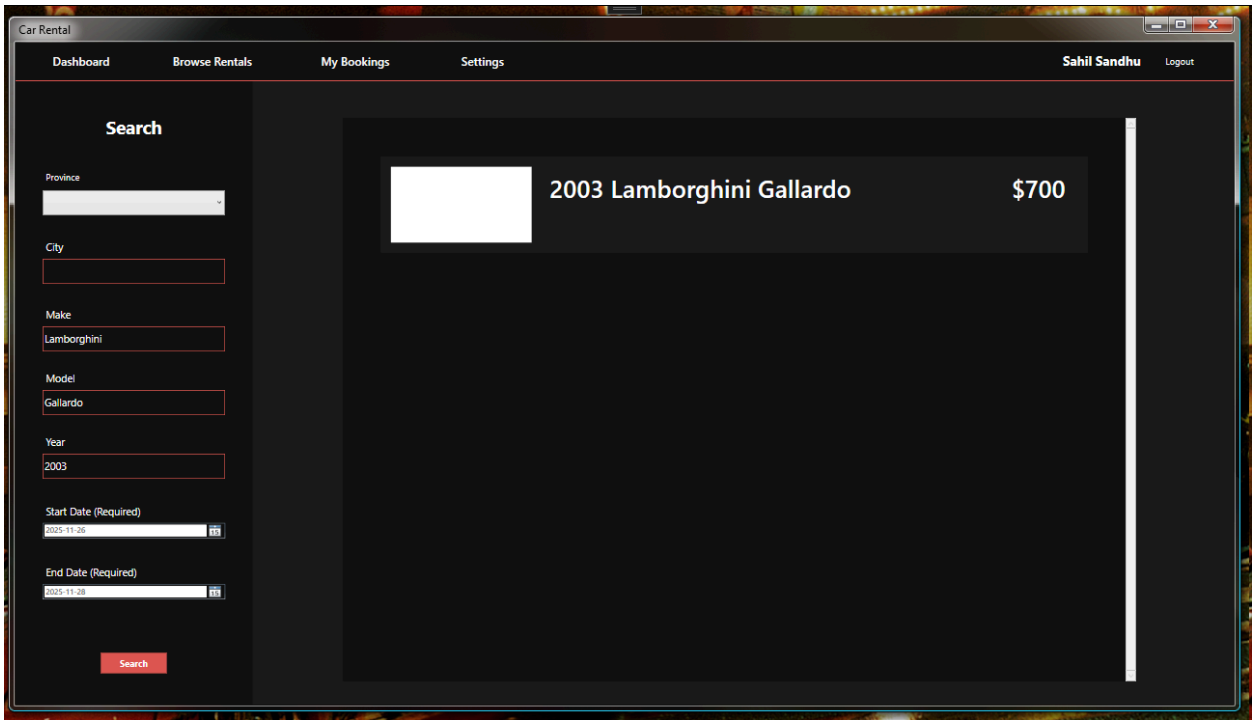
When no search filters are active (except for date), it shows all of the cars that are available and the cars that are available before the user's start date.



Search with only Province, City and a start date set to before some other bookings end.



Search with no filters but with a Start Date after every booking is available, it shows all cars in the table.



Search with Make, Model, and Year.

Challenges and Solutions:

Difficulties with the Rental Business Rules

- Setting minimum and maximum rental terms
- Managing the scheduling of automotive maintenance
- Overseeing pricing policies and discounts
- Processing refunds and cancellations

Fixes

- Put the business rule validation service into practice.
- Build a system for scheduling maintenance.
- Construct a flexible pricing engine.
- Use the logic of the cancellation policy.

Challenges in the User Experience

- delivering real-time vehicle availability updates
- Resolving issues with form validation and user input
- Creating a user interface that adapts to different screen sizes
- Using navigation flows that are easy to use

Fixes

- Use data binding while following the appropriate validation protocols.
- Implement comprehensive input validation.
- Create responsive, adjustable-sized WPF layouts.
- Make patterns for navigation that are easy to understand.

Problems with Application Performance

- Large vehicle lists can be loaded without the user interface freezing.
- Effective caching and loading of images
- Controlling memory consumption when working with big data sets
- Improving the performance of database queries

Fixes

- Put virtualized lists into practice for big data.
- Employ compression and image caching.
- Adopt appropriate disposal procedures
- Include indexing and database query optimization.

Deferred to Future Sprints

All user stories were implemented and tested in this sprint. There are no user stories deferred to future sprints.

Testing and Review

We verified functionality using user tests and user simulation. During development, once a module was done, an example being the login page, it was thoroughly tested before the development was moved on to another module. When the entire sprint was completed, we simulated how a user would navigate through the sign in/login process, to the search functionality.

Story ID	Story Title	Card	PASS/FAIL
AUTH-1	Secure Account Creation	"As a new user, I want to create an account with my email and a secure password so that I can access the platform's features."	PASS
AUTH-2	Secure Login & Logout	"As a registered user, I want to log in and out securely so that my personal information and booking history are protected."	PASS
UI-1	Vehicle Search	"As a customer, I want to search for available cars by date and location on the homepage so that I can quickly see my options."	PASS
UI-2	Vehicle Search Result Basic Information	"As a customer, I want to see a list of available cars with key details (image, model, price) after searching so that I can compare my choices."	PASS

https://drive.google.com/file/d/1pXKWlx49Abo_QQ4NdWHjMmqqUj5Wc8AT/view?usp=sharing